System Design — Full Architecture

Designer–Manufacturer Marketplace Platform | Microservices + Redis + Docker + Razorpay Route

1. Client Layer

Web App
Next.js — manufacturer, designer & admin dashboards, booking, bidding, chat UI

Mobile (future)
React Native / PWA wrapper on same API layer

↓

2. Edge / CDN

Vercel Edge Network
Serves Next.js frontend, static assets, image optimization, edge caching

Cloudflare (optional)
DDoS protection, additional CDN layer, WAF rules for API abuse

↓

3. API Gateway / Load Balancer

API Gateway
Single entry point — routes requests to correct microservice, handles auth token validation, rate limiting

Load Balancer
Distributes traffic across containerized service instances (built-in on hosting platform)

↓

4. Microservices (Dockerized Containers)

Each service is an independently deployable Docker container — can scale, deploy, and fail independently.

Auth Service
Login, signup, roles (manufacturer/designer/admin), JWT/session issuance

Profile Service
Designer & manufacturer profiles, portfolios, availability, tiers

Curation Service
Approval queue, interview scheduling status, curator decisions, NDA tracking

Booking Service
Direct-hire bookings, staged payment state machine, contact-reveal logic

Bidding Service
Project listings, bid submissions, listing fee status, monthly caps

Payment Service
Wraps Razorpay Route calls, staged split logic (token/milestone/final), commission calc

Chat Service
In-app messaging post-payment, message history, moderation hooks

Notification Service
Email (Resend) + in-app + push triggers on key events

AI Automation Service
Wraps LLM API — portfolio pre-screen, shortlist matching, support triage, dispute summaries

Admin Service
Flag/ban actions, transaction views, dispute resolution panel

↓

5. Caching & Async Layer

Redis — Cache
Session storage, designer search/shortlist results cache, rate limiting counters, monthly bid/post caps

Redis — Pub/Sub
Real-time chat presence & delivery, live notification push between services

Redis — Job Queue (BullMQ)
Background jobs: payment webhook processing, AI scoring jobs, email sending, bidding-window auto-close

↓

6. Data Layer

PostgreSQL (Supabase)
Primary relational database — all core entities per ER model (users, bookings, payment stages, bids, etc.)

Supabase Storage
Portfolio files, NDA documents, tech packs, chat attachments

Supabase Auth
Underlying identity store used by Auth Service

↓

7. External Integrations

Razorpay Route
Split payments — platform commission vs. designer payout, staged transfers, sub-accounts per designer

Claude / OpenAI API
Powers AI Automation Service — matching, scoring, summarization

Resend
Transactional email — approvals, payment confirmations, bid updates

Cal.com
Designer interview scheduling for curator/admin team

↓

8. Hosting & DevOps

Vercel
Hosts Next.js frontend + edge functions

Railway / Render (or AWS ECS later)
Hosts Dockerized microservices + Redis instance

Docker
Each microservice packaged as its own container image, docker-compose for local dev

GitHub Actions
CI/CD — build, test, deploy containers on push

Sentry + Logs
Error tracking and centralized logging across all services

Client Edge/CDN Gateway Microservices Cache/Queue Data External APIs Infra/DevOps

Key Request Flows Through the Architecture

How the layers above interact for the platform's most important actions

Flow A — Manufacturer Pays Booking Token

- Client (Next.js) → API Gateway → Booking Service
- Booking Service → Payment Service → Razorpay Route (create order)
- Razorpay webhook → API Gateway → Payment Service (async job queued in Redis/BullMQ)
- Payment Service confirms split → writes to PostgreSQL (Transactions, Payment Stages)
- Booking Service updates`contact_reveal_status`→ Redis Pub/Sub notifies Chat Service to unlock chat
- Notification Service sends confirmation email (Resend) + in-app alert to both parties

Flow B — Designer Submits Portfolio for Approval

- Client → API Gateway → Profile Service (saves portfolio to Supabase Storage)
- Profile Service → Curation Service (creates review queue entry)
- Curation Service → AI Automation Service (async job via Redis queue) → pre-screen score written back
- Curator/Admin reviews via Admin Service → approves/rejects → writes decision to PostgreSQL
- Notification Service informs designer of outcome

Flow C — Manufacturer Posts a Bidding Project

- Client → API Gateway → Bidding Service (checks Redis for monthly cap)
- Bidding Service → Payment Service → Razorpay (listing fee, e.g. ₹99)
- On payment success → listing goes live, cached in Redis for fast designer-side browse queries
- Redis job scheduled to auto-close listing at bidding window end (BullMQ delayed job)
- On window close → Notification Service alerts manufacturer to select a bid

Why This Piece Is Here	Reasoning
Redis (cache)	At 500 users/day this isn't strictly required yet, but pays off fast for designer search/shortlist queries and monthly-cap checks that would otherwise hit Postgres repeatedly.
Redis (queue/BullMQ)	Payment webhooks, AI scoring calls, and scheduled jobs (bidding window auto-close) should never block the user-facing request — queue them and process async.
Docker + Microservices	Lets you deploy/scale the Payment Service or Chat Service independently without redeploying the whole app — useful once traffic grows unevenly across features.
Razorpay Route	Only integration point actually required — handles the split commission and staged transfers per booking.
Honest scale note	At 500 users/day, a full microservices split is heavier than strictly necessary — a modular monolith (single Next.js app, cleanly separated modules) would ship faster. This diagram shows the microservices version since that's what you asked for, but flag this trade-off with your developer before committing to it.